

Interlude: Modules as Legibility Boundaries

Opening Question

If code exists inside a program but cannot be named or called from another region, in what sense is it available there?

1. A program needs internal boundaries

Picture a single file whose functions and types have accumulated, over time, into one undifferentiated namespace. Anything can call anything; any type's fields are as visible from the bottom of the file as from directly beside them. This is not primarily a tidiness problem, though it is often described as one. The deeper problem is that every internal detail has become equally legible to every reader and every caller, whether or not that detail was ever meant to be depended upon. Nothing in the file distinguishes "this exists to support something else" from "this is the thing you are meant to use."

2. Declaring a module

Rust lets a program declare an internal boundary directly, without moving anything to a separate file first.

```
mod ledger {  
    fn verify_total() {  
        // internal mechanism  
    }  
  
    pub fn submit() {
```

```
    verify_total();  
  }  
}
```

The `mod` block establishes a named region, `ledger`, with its own internal reachability structure. Inside that region, `verify_total` and `submit` can see and call each other freely, exactly as they could in the undifferentiated file above. What changes is what code *outside* the block can reach. By default, nothing declared inside a module is visible from outside it at all.

3. Privacy is not secrecy

It is worth being precise about what a private item is and is not. `verify_total` above still exists; it still runs; it still affects the observable behavior of `submit`. Anyone holding the source can read it in full. Privacy in Rust does not encrypt or conceal an implementation from a reader who possesses the code, and it is not a security mechanism in that sense at all. What it controls is narrower and more specific: which distinctions are directly *operational* from a given point in the program — which names can be written in a call, a field access, or a pattern, and have the compiler accept them. A private function is fully present and fully consequential; it is simply not one of the things another module is permitted to name.

4. `pub` as exported continuation

Marking an item `pub` does the opposite of nothing: it creates a path through which code outside the module may continue into it. Before `submit` was marked `pub` in the example above, no code outside `ledger` had any way to trigger `ledger`'s behavior at all. After it, outside code gained exactly one such path, and no more — `verify_total` remains as unreachable from outside as it was before. Every `pub` marking is therefore a deliberate enlargement of the module's external state-transition surface, one item at a time, and the size of that surface is a fact a reader can determine just by scanning for the keyword.

5. Restricted visibility

Visibility is not simply a binary choice between fully public and fully private. Rust permits intermediate widths, granted relative to a program's own

structure rather than to the outside world in general.

```
pub(crate) fn audit_log() {
    // visible anywhere in this crate, not outside it
}
```

`pub(crate)` opens a path to every other module within the same crate while leaving the item completely closed to anything outside that crate. `pub(super)` narrows the grant further, to the immediately enclosing module alone. These are not weaker versions of `pub`; they are answers to a different question — not *whether* an item is reachable from elsewhere, but *how far* elsewhere is permitted to extend.

6. Paths and use

Reaching an item through a module boundary requires naming a path to it.

```
crate::ledger::submit();

// or, after bringing the name into local scope:
use crate::ledger::submit;
submit();
```

`crate`, `self`, and `super` anchor a path at the crate root, the current module, or the immediately enclosing module, respectively; a path may also simply begin at the item’s home module and descend from there. `use` does not create a second copy of `submit`, move it, or change where it lives. It is purely a local change in legibility — an abbreviation that lets later code write `submit()` instead of the fully qualified path, without altering the underlying ownership or location of the item one bit.

7. Files do not define the theory

A module declared with `mod ledger;` (no body) typically corresponds to a file, `ledger.rs`, or a directory, `ledger/mod.rs` or `ledger/` with an equivalent layout, and larger programs are usually organized this way rather than with everything nested in one file behind `mod` blocks. It is tempting to think of the module tree as simply *being* the filesystem tree, but the two are related, not identical. The filesystem layout supports the module tree by giving the compiler a place to look for each module’s source; the actual visibility relations — what is `pub`, what is `pub(crate)`, what remains private

— are declared inside the files themselves and would mean the same thing if the files were laid out differently. Reading a program’s module structure from its directory listing alone tells you what exists; it does not by itself tell you what is reachable from where.

8. Private fields and guarded construction

Chapter 6 introduced structs whose constructors enforce an invariant on construction rather than trusting every field to be set validly by hand. Module privacy is what makes that enforcement actually hold once code leaves the module where the struct is defined.

```
pub struct Percentage {
    value: f64, // private: not settable directly from outside
}

impl Percentage {
    pub fn new(value: f64) -> Option<Self> {
        if (0.0..=100.0).contains(&value) {
            Some(Percentage { value })
        } else {
            None
        }
    }
}
```

Outside code can hold a `Percentage`, pass it around, and read whatever public methods it offers, but cannot write `Percentage { value: 500.0 }` directly, because `value` is private. The public struct name and the private field together permit outside code to possess and use a value without ever being able to construct one of the theoretically possible internal states — like `500.0` — that the constructor was written to rule out. Privacy is what keeps the guarantee from Chapter 6’s constructor from being bypassable the moment the type crosses a module boundary.

9. Re-export as controlled reconstruction

A module’s internal organization need not match the shape it presents to the outside world. `pub use` lets a module re-export an item from somewhere else inside it, at a path chosen independently of where the item actually lives.

```
mod internal {
    pub struct Report;
}

pub use internal::Report;
```

Outside code can now write `crate::Report` directly, with no knowledge that `Report` is actually defined three levels down inside `internal`. The exported path is a deliberately constructed public view, free to diverge from the internal topology that produced it — which means the internal organization can later be rearranged, split, or renamed, without disturbing the external path anything outside the crate has already come to depend on.

10. Boundary design

Compare two versions of the same module: one that marks every helper function `pub` because it was easiest not to think about it, and one that exposes a small, deliberately chosen operational surface while keeping every supporting function `private`. Both versions may contain the identical internal capability. What differs is where valid use is permitted to happen. The narrower interface does not reduce what the module can do; it concentrates the ways outside code is allowed to make it do those things into a small, legible, and — because the rest is unreachable by construction rather than by convention — enforceable set of paths.

Invariant Established

A module determines which internal distinctions become legible and actionable from elsewhere in the program. Its public interface is a designed boundary of continuation, not incidental leftover from where the source happens to live on disk.

Non-overlap note: Chapter 6 explains how types constrain construction, and Chapter 11 explains composition over inheritance. This interlude explains the Rust syntax and topology — `mod`, `pub`, `paths`, and `use` — by which those constraints are distributed and enforced across a program's own module structure.